

UNIVERSIDADE ESTADUAL DO PIAUÍ

UESPI | Educação a Distância

Curso de Tecnologia em Sistemas para Internet

Polo Jerumenha

ConnectNet Entrega 3

Projeto Back End

Sistema de Gerenciamento de Provedor de Internet

Discentes

Adrielly Ferraz de Oliveira Brito

Caroline Borges Albuquerque

Lusia Alves da Silva Sousa Neta

Terceira etapa de desenvolvimento

API | Interface | Persistência | Versionamento

Jerumenha PI

09 de outubro de 2026

1 Objetivo da terceira entrega

A terceira etapa do ConnectNet consolida uma API REST para a gestão de clientes, planos, faturas e chamados e uma interface integrada a esses serviços. O pacote reúne o código executável, o adaptador de persistência MySQL, os testes, a documentação e 16 tarefas organizadas para a revisão do grupo no Trello.

A base tecnológica acompanha a Entrega 2: Node.js, Express, MySQL e interface em HTML, CSS e JavaScript nativo. A validação com MySQL 8.0.46 aprovou a API, a interface e a permanência dos registros após reinícios. Para concluir a etapa acadêmica, faltam a publicação da versão no repositório remoto e o registro e a revisão dos cartões no Trello.

Requisito da atividade	Artefato da etapa
Definir as tecnologias do back end	Configuração e dependências da API documentadas.
Criar ou manter repositório remoto	Roteiro de integração à branch da Entrega 3 no GitHub.
Versionar a API e integrar ao remoto	API organizada em backend e fluxo Git de publicação.
Implementar ao menos 8 tarefas da API	8 tarefas API01 a API08 com arquivos e critérios.
Implementar ao menos 8 tarefas da UI	8 tarefas UI01 a UI08 com arquivos e critérios.

2 Tecnologias definidas

Camada	Tecnologia e uso
Execução	Node.js 22 ou 24. Validação local com Node.js v24.19.0.
Serviços HTTP	Express 5.2.1 para rotas, JSON, arquivos estáticos e erros.
Persistência	MySQL 8.0.46 validado; driver mysql2 3.24.5 com consultas parametrizadas.
Senhas e sessões	Scrypt do módulo crypto; cookie HttpOnly e sessão no servidor.
Interface	HTML5, CSS3 e JavaScript nativo; integração assíncrona por fetch.
Controle de versão	Git e GitHub; dependências fixadas no package-lock.json.
Organização das tarefas	Trello com cartões separados para API e UI.

React foi previsto na Entrega 1. A continuação mantém a escolha de JavaScript nativo consolidada na Entrega 2.

3 Organização da API e do banco

O servidor Express disponibiliza a interface e os serviços sob /api/v1 na porta 3000. As rotas aplicam autenticação e permissões; os serviços concentram as regras; os repositórios separam o acesso aos dados. No modo demo, os dados ficam na memória. No modo mysql, são gravados no banco connectnet_entrega3.

Tabela	Função e vínculo principal
plano	Nome, velocidade, preço e descrição do serviço.
cliente	Cadastro, situação e plano contratado; vínculo com plano.
usuario	E-mail, senha_hash e perfil; cliente_id para contas de clientes.
fatura	Valor, emissão, vencimento, situação e competência; vínculo com cliente.
chamado	Descrição, abertura, situação e prioridade; vínculo com cliente.

A tabela usuario separa o acesso das informações cadastrais. Uma restrição única em fatura impede duplicidade por cliente e competência. Faturas e chamados preservam o histórico e impedem a exclusão de clientes vinculados; a desativação continua disponível.

3 1 Perfis de acesso

Perfil	Permissões
Administrador	Gerencia todos os módulos e os registros.
Suporte	Consulta clientes e planos; abre e atualiza chamados; não acessa faturas nem exclui registros.
Cliente	Consulta seu plano e suas faturas; abre e acompanha apenas seus chamados.

3 2 Rotas principais

Módulo	Endpoints sob api v1
Sessão	POST /auth/login; GET /auth/me; POST /auth/logout.
Clientes e planos	GET e POST /clientes e /planos; GET, PUT e DELETE por /{id}.
Faturas	GET e POST /faturas; GET, PUT e DELETE /faturas/{id}; POST /faturas/gerar.
Chamados	GET e POST /chamados; GET, PUT e DELETE /chamados/{id}.
Operação	GET /saude; GET /dashboard.

Respostas de sucesso usam dados; erros usam erro com código e mensagem. Escritas autenticadas exigem token CSRF. Credenciais incorretas, acesso indevido, duplicidade e vínculos inválidos possuem respostas HTTP específicas.

4 Tarefas da API

Os oito cartões abaixo correspondem a entregas identificáveis no código. O arquivo TAREFAS_TRELLO.md contém as descrições completas, dependências, caminhos e checklists para o quadro. A execução com MySQL faz parte do aceite de API02.

ID	Tarefa	Critério de aceite
API01	Configurar o projeto e o servidor	Servidor inicia na porta configurada e GET /api/v1/saude retorna 200.
API02	Preparar a persistência MySQL	Executar npm run db:setup em banco novo, iniciar DB_MODE=mysql e confirmar persistência após reinício.
API03	Implementar autenticação e perfis	Login válido cria cookie HttpOnly; credenciais incorretas e operações sem permissão são recusadas.
API04	Implementar o cadastro de clientes	Cadastro cria cliente e usuário juntos; duplicidade desfaz a operação e desativação impede novo acesso.
API05	Implementar a gestão de planos	Preço positivo é obrigatório; plano com clientes vinculados não pode ser excluído.
API06	Implementar o faturamento mensal	Geração considera apenas clientes ativos; repetição não duplica cobrança; pagamento altera os totais.
API07	Implementar os chamados de suporte	Cliente acessa somente seus chamados; suporte atualiza situação e prioridade sem excluir registros.
API08	Validar dados e documentar a API	npm test passa; dados inválidos retornam 400, falta de sessão 401, permissão 403, ausência 404 e conflito 409.

Código entregue: backend/server.js, backend/src, backend/scripts, database/schema.sql, tests e docs/openapi.json. Foram aprovados 21 testes HTTP em demo, os mesmos 21 em MySQL e mais 9 testes SQL. A evidência de API02 está em VALIDACAO_MYSQL.md e validacao-mysql.json.

5 Tarefas da interface

Os oito cartões da UI cobrem o acesso, a navegação e os módulos do projeto. As telas utilizam respostas reais da API e apresentam as ações permitidas para o perfil autenticado. A autorização também é aplicada no servidor.

ID	Tarefa	Critério de aceite
UI01	Construir a tela de login	Campos possuem rótulos; falha de login aparece na tela; acesso válido abre o painel.
UI02	Construir o painel por perfil	Administrador vê a gestão geral; cliente vê seu plano e registros; suporte vê a central de chamados.
UI03	Construir a gestão de clientes	Cadastro e edição aparecem na lista; busca filtra os registros; senha é opcional durante edição.
UI04	Construir a gestão de planos	Plano salvo aparece na lista; edição altera a mensalidade; exclusão apresenta a resposta da API.
UI05	Construir a gestão de faturas	Nova fatura aparece na lista; marcar paga atualiza situação; cliente recebe apenas suas cobranças.
UI06	Construir a central de chamados	Chamado aberto aparece na lista; suporte ou administrador altera status; cliente acompanha sua solicitação.
UI07	Integrar a interface à API e à sessão	Painel sem sessão redireciona; logout encerra acesso; operações usam respostas reais da API.
UI08	Conferir responsividade e mensagens	Tela de 390 px não causa rolagem horizontal da página; tabelas mantêm rolagem própria; HTML de usuários permanece texto.

Código entregue: frontend/login.html, frontend/index.html, frontend/js e frontend/css. As verificações da interface exercitaram os três perfis, os formulários dos módulos, os filtros e a tela com largura de 390 pixels.

6 Execução e validação

6 1 Demonstração local

Abra a pasta do pacote no VS Code. Duplique .env.example como .env e mantenha DB_MODE=demo. No terminal da pasta que contém package.json, execute:

```
npm ci
npm start
```

Acesse <http://127.0.0.1:3000/login.html>. As contas fictícias abaixo usam a senha ConnectNet#2026. Reiniciar o servidor restaura os exemplos do modo demo.

Perfil	E-mail
Administrador	admin@connectnet.local
Suporte	suporte@connectnet.local
Cliente	cliente@connectnet.local

6 2 Resultados em 09 de outubro de 2026

Verificação	Resultado e alcance
API HTTP	21 testes aprovados em demo e 21 em MySQL; nenhuma falha.
Integridade MySQL	9 testes SQL aprovados: vínculos, unicidade, CHECK, rollback, tipos e reconexão.
Interface no navegador	15 verificações em demo e 15 em MySQL; zero erros JavaScript. Painel móvel de 390 pixels aprovado.
Persistência	Plano, cliente, fatura, chamado e login preservados após reiniciar Node e, depois, mysqld e Node.
Servidor e estrutura	MySQL 8.0.46 real; cinco tabelas InnoDB com utf8mb4 e rota de saúde em modo mysql.
Evidências remotas	Publicação do código no GitHub e registro e revisão no Trello ainda pendentes.

A validação usou dados fictícios em ambiente isolado. npm test verifica a API em demo; npm run test:mysql executa os 30 testes com MySQL. A UI usa npm run test:ui ou npm run test:ui:mysql após instalar o Chromium do Playwright. Consulte VALIDACAO_MYSQL.md para repetir os procedimentos.

6 3 Executar com persistência MySQL

Inicie o servidor de banco, ajuste as credenciais no .env e mantenha DB_NAME=connectnet_entrega3. Execute npm run db:setup; depois altere DB_MODE=mysql e reinicie a aplicação. Cadastre um registro, reinicie o servidor e confira sua permanência. O setup preserva os registros existentes e não migra o banco anterior.

7 Evidência local da interface

A captura abaixo mostra a aplicação com o perfil administrador e persistência MySQL. A execução ocorreu em 09/10/2026, em ambiente isolado com dados fictícios. Os números apresentados são exemplos para validação das telas.

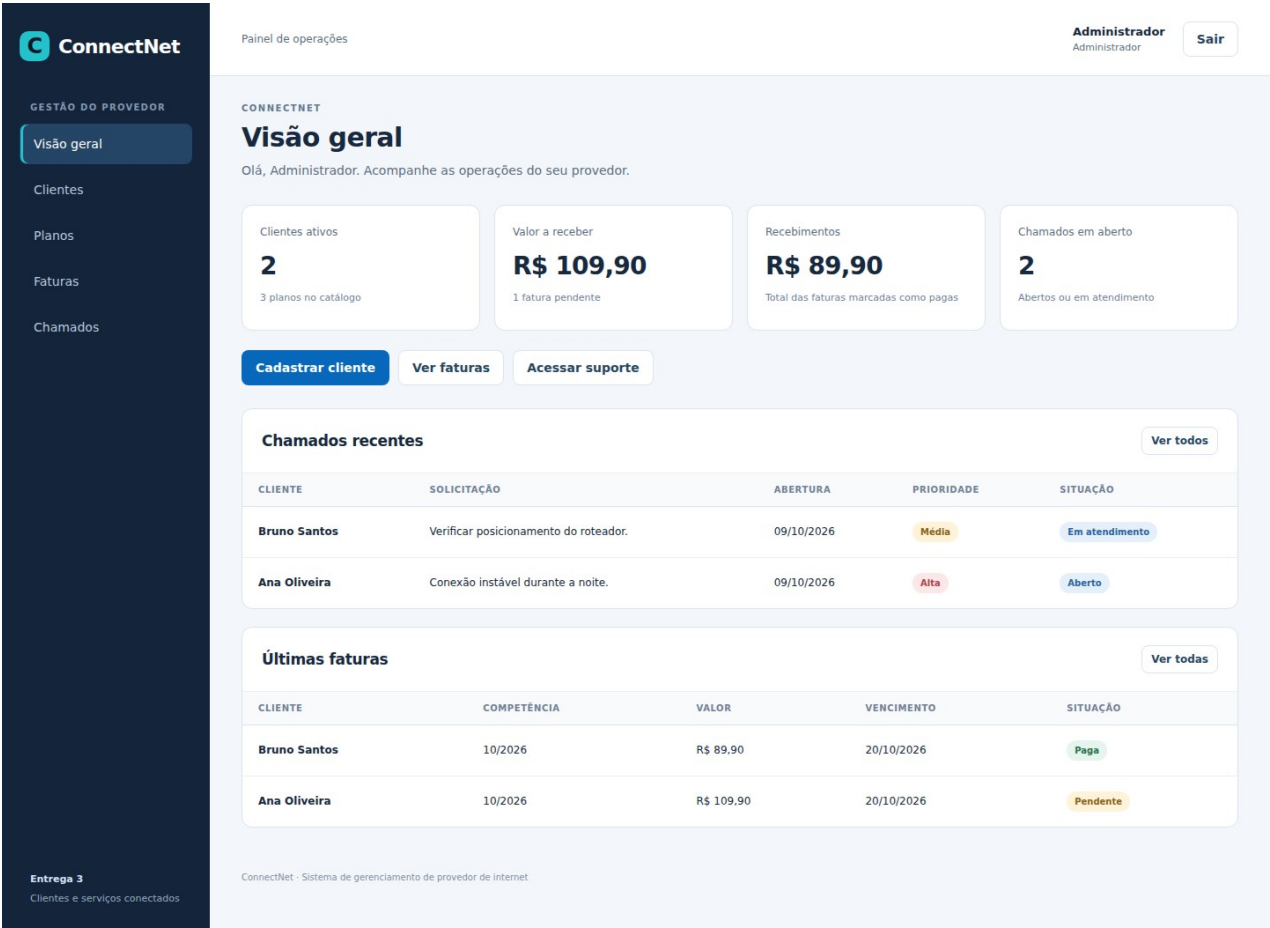


Figura 1 Painel administrativo com métricas e listas de chamados e faturas

7 1 Capturas incluídas no pacote

Arquivo	Conteúdo verificado
01-login-mysql.png	Tela de acesso sem atalhos de contas demo.
02-painel-admin-mysql.png	Painel administrativo conectado ao MySQL.
03-chamados-admin-mysql.png	Chamado cadastrado e atualizado para resolvido.
04-painel-cliente-mobile-mysql.png	Painel do cliente na largura de 390 pixels.

validacao-mysql.json comprova as tabelas, as restrições e os registros após os reinícios. mysql-api-integridade.tap contém os 30 testes. Acrescente as evidências remotas de GitHub e Trello após essas ações.

8 Versionamento e conclusão da etapa

A Entrega 2 indica o repositório <https://github.com/luis2alves/ConnectNet> e o quadro <https://trello.com/b/iS5eRal6>. A versão desta etapa deve ser integrada ao repositório efetivo do grupo. O arquivo GIT_E_ENTREGA.md descreve a comparação dos arquivos, os comandos e as evidências de conclusão.

8 1 Fluxo de integração

Obtenha a cópia atual do repositório, crie a branch entrega3 e compare os arquivos com este pacote. Após integrar as alterações, execute os testes nessa cópia. Registre o commit e envie a branch para o remoto. Acrescente o link efetivo da versão ao relatório e aos cartões.

```
git switch -c entrega3
git add .
git commit -m "Entrega 3: API, interface, testes e documentação"
git push -u origin entrega3
```

Se a branch já existir, use git switch entrega3. O .gitignore exclui .env e node_modules; as dependências são reconstruídas por npm ci.

8 2 Organização e comprovação no Trello

Cadastre os cartões API01 a API08 e UI01 a UI08, defina as responsáveis e anexe os critérios de aceite. Use as listas A fazer, Em andamento, Em revisão e Concluído. Cada conclusão deve remeter a código publicado e evidência de execução; API02 também exige a validação com MySQL.

Evidência de conclusão	Registro a acrescentar
Código remoto	Link do commit ou da branch e captura das pastas API e UI.
Trello	Quadro com 8 cartões API e 8 UI e cartões revisados.
MySQL	Validação local concluída; anexar VALIDACAO_MYSQL.md e validacao-mysql.json ao cartão API02.
Relatório	Atualizar os registros acima no documento editável.

O título da atividade informa o período de 14/09/2026 a 10/10/2026; a disponibilidade mostrada termina em 11/10/2026 às 23h59. O grupo deve organizar o envio até 10/10 e conferir o encerramento no ambiente virtual.

9 Referências

CONNECTNET. Entregas 1 e 2. Relatórios acadêmicos do grupo, 2026.

UESPI. Entrega 3 Projeto Back End. Instruções da atividade no ambiente virtual, 2026.

EXPRESS. Routing. <https://expressjs.com/en/guide/routing/>. Acesso em 08 out. 2026.

NODE.JS. Crypto. <https://nodejs.org/api/crypto.html>. Acesso em 08 out. 2026.

MYSQL2. Quickstart. <https://sidorares.github.io/node-mysql2/docs>. Acesso em 08 out. 2026.